



Instituto Superior de Ciências e Tecnologia de Moçambique

Escola de Engenharias e Tecnologia

Curso de Engenharia Informática

Disciplina de Inteligência Artificial

RELATÓRIO DE PROJECTO DE INTELIGÊNCIA ARTIFICIAL

TurRoute — Planeamento de Circuitos Turísticos em Moçambique

Grupo 1 — Tema 4.2.8

Estudantes:

20240143 — Aashir Omar

20240459 — Igor dos Santos

20240567 — Stanley Cossa

20240526 — Quirson Ngale

20220057 — Saudah Ahmad

20230414 — Hélder Júnior

Docente:

Ruben Manhiça

Maputo, Maio de 2026

1. Introdução

O presente relatório documenta o desenvolvimento do **TurRoute**, um sistema inteligente de optimização de circuitos turísticos em Moçambique. O sistema foi desenvolvido no âmbito da disciplina de Inteligência Artificial do ISCTEM, com o objectivo de resolver o Problema do Caixeiro Viajante (TSP — Travelling Salesman Problem) aplicado à rede de voos domésticos da LAM — Linhas Aéreas de Moçambique.

O problema proposto consiste em determinar a ordem óptima de visita a um conjunto de cidades, minimizando a distância total percorrida. Apesar de simples na enunciação, o TSP pertence à classe de problemas NP-difícil: com 11 cidades, existem 3 628 800 rotas possíveis, tornando a verificação exhaustiva computacionalmente inviável.

A solução implementada combina quatro algoritmos de busca — Greedy, BFS, DFS e A* com heurística MST — numa aplicação web completa com mapa interactivo, animação de rotas e comparação visual de desempenho.

2. Descrição do Problema

2.1 Enunciado (Tema 4.2.8)

O tema 4.2.8 propõe o desenvolvimento de um sistema para planear uma viagem entre várias cidades, inspirado no Problema do Caixeiro Viajante (TSP). O agente deve:

- Partir de uma cidade inicial definida pelo utilizador
- Visitar todas as cidades seleccionadas exactamente uma vez
- Minimizar o custo total da viagem (distância percorrida)

2.2 Requisitos funcionais e correspondência no TurRoute

Requisito	Implementação no TurRoute
Definir número de cidades	Toggles na sidebar — selecção de 2 a 11 cidades
Matriz de distâncias	Gerada automaticamente pela fórmula de Haversine a partir de coordenadas reais
Escolher cidade inicial	Dropdown "Cidade de Partida"
Executar algoritmos de busca	4 algoritmos: Greedy, BFS, DFS, A* (MST)
Mostrar cidades num plano 2D	Mapa SVG com posicionamento geográfico real de Moçambique
Representar ligações entre cidades	Arcos no mapa; badge "escala" para segmentos sem voo directo LAM
Visualizar o percurso final	Animação do avião ao longo da rota calculada
Destacar ordem de visita	Sequência numerada e ordenada na rota apresentada

2.3 Saídas produzidas pelo sistema

Para cada algoritmo executado, o sistema devolve:

Saída esperada	Localização no sistema
Ordem de visita das cidades	Rota apresentada: Maputo → Beira → ...
Custo total (km)	Métrica principal no painel de resultados
Nós explorados	Coluna na tabela de comparação + gráfico de barras
Tempo de execução	Coluna na tabela de comparação + gráfico de barras

Adicionalmente, o sistema calcula o **tempo estimado de voo** do circuito completo (à velocidade de cruzeiro da LAM, 500 km/h) e identifica automaticamente os segmentos sem voo directo na rede LAM.

2.4 Requisitos adicionais

- **Heurística admissível para A*:** implementada via Minimum Spanning Tree com o Algoritmo de Prim — demonstrada na Secção 4.4.
- **Comparação óptimo vs aproximado:** A* (óptimo garantido) comparado directamente com Greedy (heurística construtiva) — resultados na Secção 5.
- **Suporte a número elevado de cidades:** para circuitos com mais de 10 cidades, o BFS é automaticamente substituído pelo Greedy com aviso ao utilizador; o A* suporta até 12 cidades com desempenho aceitável.

3. Modelação do Problema

3.1 Representação formal

O TurRoute modela o TSP como um problema de busca num espaço de estados, com a seguinte representação formal:

Componente	Definição
Estado	Par (cidade_actual, frozenset(cidades_visitadas))
Estado inicial	(cidade_partida, {cidade_partida})
Acções possíveis	Voar para qualquer cidade ainda não visitada
Teste de objectivo	$\text{len}(\text{cidades_visitadas}) == \text{total_cidades_seleccionadas}$
Custo de passo	Distância geodésica (Haversine) entre as duas cidades
Custo total	Soma acumulada de todas as distâncias percorridas

O uso de frozenset para representar o conjunto de cidades visitadas é essencial: permite que o mesmo conjunto de cidades visitadas seja reconhecido como estado equivalente independentemente da ordem em que foram visitadas, evitando redundância na exploração.

3.2 Dados geográficos

A rede de voos é baseada nas rotas reais da LAM. As 11 cidades cobertas pelo sistema são:

Cidade	Região	Latitude	Longitude
Maputo	Sul	-25.9667	32.5833
Inhambane	Sul	-23.8650	35.3833
Vilanculos	Sul	-21.9833	35.3167
Beira	Centro	-19.8436	34.8389
Chimoio	Centro	-19.1167	33.4833
Quelimane	Centro	-17.8786	36.8881
Tete	Centro	-16.1564	33.5867
Nampula	Norte	-15.1167	39.2667
Nacala	Norte	-14.5422	40.6783
Lichinga	Norte	-13.3147	35.2392
Pemba	Norte	-12.9667	40.5167

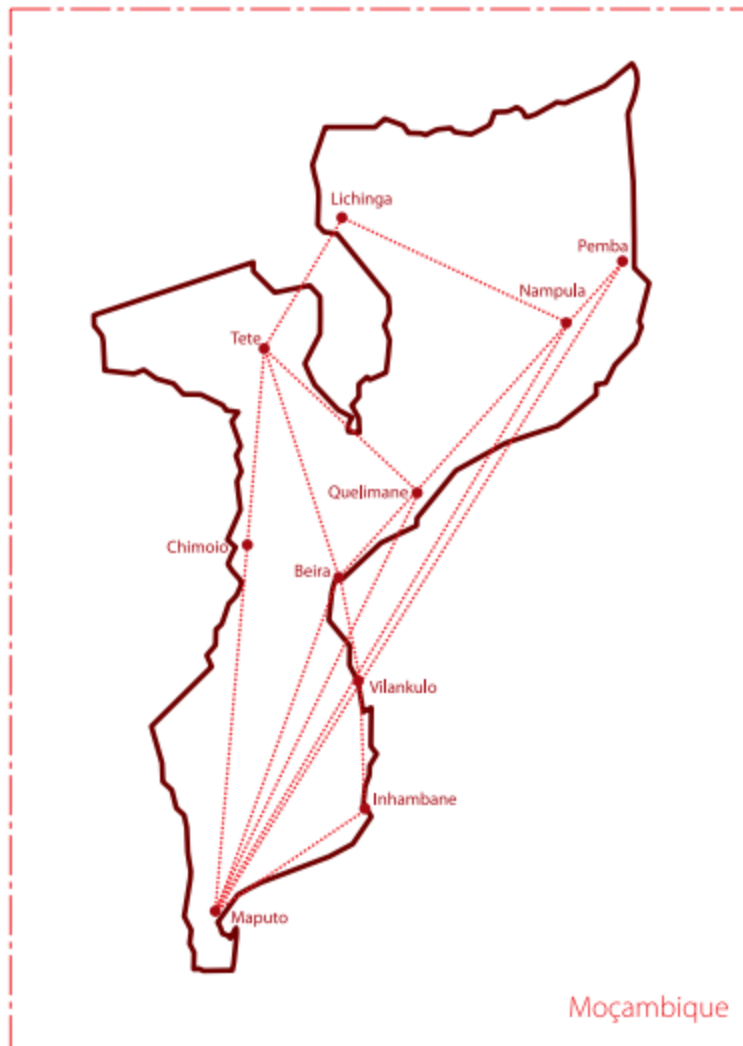


Figura 1 — Rede de rotas domésticas da LAM (fonte: lam.co.mz)

3.3 Cálculo de distâncias — Fórmula de Haversine

As distâncias entre cidades são calculadas pela fórmula de Haversine, que determina a distância real entre dois pontos na superfície terrestre a partir das suas coordenadas geográficas:

$$\begin{aligned}
 a &= \sin^2(\Delta\text{lat}/2) + \cos(\text{lat1}) \cdot \cos(\text{lat2}) \cdot \sin^2(\Delta\text{lon}/2) \\
 c &= 2 \cdot \text{atan2}(\sqrt{a}, \sqrt{1-a}) \\
 d &= R \cdot c \quad (R = 6371 \text{ km} - \text{raio médio da Terra})
 \end{aligned}$$

Esta abordagem garante precisão geográfica real, ao contrário da distância euclidiana que seria inadequada para coordenadas esféricas.

4. Algoritmos de Busca

4.1 Greedy — Vizinho Mais Próximo

Estratégia: a cada passo, selecciona a cidade não visitada mais próxima da posição actual.

Complexidade: $O(N^2)$ em tempo; $O(N)$ em espaço.

Garantia de óptimo: não. O Greedy é uma heurística construtiva. Pode chegar a soluções 20–30% piores que o óptimo em casos desfavoráveis, porque decisões localmente óptimas podem comprometer o percurso global.

Implementação:

```
def greedy(problem):
    current = problem.initial_state
    visited = {current.city}
    path = [current.city]
    total_cost = 0

    while len(visited) < len(problem.cities):
        nearest = min(
            (c for c in problem.cities if c not in visited),
            key=lambda c: problem.distance(current.city, c)
        )
        total_cost += problem.distance(current.city, nearest)
        current = State(nearest, frozenset(visited | {nearest}))
        visited.add(nearest)
        path.append(nearest)

    return path, total_cost
```

Uso recomendado: circuitos com muitas cidades onde velocidade de resposta é prioritária sobre óptimo absoluto.

4.2 BFS — Busca em Largura

Estratégia: explora todos os estados ao mesmo nível de profundidade antes de avançar para o seguinte. Usa uma fila FIFO.

Complexidade: $O(N!)$ em tempo e espaço — cresce factorialmente com o número de cidades.

Garantia de óptimo: sim, em número de passos. Com distâncias variáveis, o BFS não garante a rota de menor custo — apenas a de menor número de voos. Para custo óptimo, o A^* é o algoritmo correcto.

Limitação implementada: o TurRoute bloqueia o BFS para circuitos com mais de 10 cidades, substituindo automaticamente pelo Greedy com aviso ao utilizador. Para 11 cidades, o BFS teria de explorar potencialmente milhões de estados, tornando-o impraticável.

4.3 DFS — Busca em Profundidade

Estratégia: explora um caminho até ao fim antes de recuar e tentar outro. Usa uma pilha (LIFO).

Complexidade: $O(N!)$ em tempo; $O(N)$ em espaço — mais eficiente em memória que o BFS.

Garantia de óptimo: não. O DFS explora sem critério de qualidade, podendo percorrer ramos de custo elevado antes de encontrar soluções melhores.

Nota de implementação: os resultados do DFS podem variar entre execuções devido ao comportamento de iteração sobre frozenset em Python, que não garante ordem determinista.

Resultado empírico com 11 cidades: 23 051 estados explorados, rota de 5 524 km — 87% pior que o óptimo.

4.4 A* com Heurística MST — Busca Informada

Este é o algoritmo principal do sistema — o único que garante a rota óptima de forma eficiente.

Estratégia: expande sempre o estado com menor valor $f = g + h$, onde:

- g = custo real acumulado (quilómetros já voados)
- h = estimativa do custo mínimo restante

Usa uma fila de prioridade (heap mínimo) para seleccionar eficientemente o estado mais promissor.

A heurística — Minimum Spanning Tree (MST):

Para estimar o custo restante, o A* calcula a Árvore de Cobertura Mínima das cidades ainda não visitadas, usando o **Algoritmo de Prim**:

```
def mst_heuristic(unvisited_cities, distances):
    if len(unvisited_cities) <= 1:
        return 0
    cities = list(unvisited_cities)
    in_tree = {cities[0]}
    min_cost = 0

    while len(in_tree) < len(cities):
        edge = min(
            (distances[u][v] for u in in_tree for v in cities if v not in
in_tree),
            default=0
        )
        min_cost += edge
        for u in in_tree:
            for v in cities:
                if v not in in_tree and distances[u][v] == edge:
                    in_tree.add(v)
                    break

    return min_cost
```

Por que é admissível: a MST representa o custo mínimo absoluto de conectar todas as cidades restantes. Qualquer rota real que as visite tem de as conectar de alguma forma — e nunca pode fazê-lo com um custo inferior à MST. Logo, h nunca sobrestima o custo real.

Teorema: uma heurística admissível garante que o A* encontra sempre a solução ótima.

Resultado empírico com 11 cidades: apenas 21 estados explorados, rota ótima de 2 951 km.

4.5 Comparação dos algoritmos

Critério	Greedy	BFS	DFS	A* (MST)
Garante óptimo	Não	Sim (passos)	Não	Sim (custo)
Complexidade temporal	$O(N^2)$	$O(N!)$	$O(N!)$	$O(N^2 \cdot 2^N)$
Complexidade espacial	$O(N)$	$O(N!)$	$O(N)$	$O(N \cdot 2^N)$
Estrutura de dados	lista	fila FIFO	pilha	heap mínimo
Limite prático	sem limite	≤ 10 cidades	500 000 nós	≤ 12 cidades

5. Resultados Experimentais

5.1 Cenário 1 — Circuito Sul (4 cidades)

Cidades: Maputo, Inhambane, Vilanculos, Beira — **Partida:** Maputo

Algoritmo	Rota encontrada	Custo (km)	Tempo de voo	Nós explorados
A* (MST)	Maputo → Inhambane → Vilanculos → Beira	819,8	3h 05min	4
Greedy	Maputo → Inhambane → Vilanculos → Beira	819,8	3h 05min	4
BFS	Maputo → Inhambane → Vilanculos → Beira	819,8	3h 05min	16
DFS	Maputo → Inhambane → Beira → Vilanculos	1 063,0	3h 10min	16

Neste cenário, o Greedy encontrou casualmente o ótimo — não é comportamento garantido.

5.2 Cenário 2 — Circuito Completo (11 cidades)

Todas as cidades da rede LAM — Partida: Maputo

Algoritmo	Custo (km)	Tempo de voo	Nós explorados
A* (MST)	2 951	9h 14min	21
Greedy	3 165	8h 20min	11
BFS	— (bloqueado: >10 cidades)	—	0
DFS	5 524	14h 6min	23 051

Tempo de voo inclui o regresso à cidade de partida, à velocidade de cruzeiro da LAM (500 km/h).

5.3 Análise dos resultados

- O A* explorou **1098× menos estados** que o DFS e encontrou uma rota **47% mais curta**.
- O Greedy oferece um compromisso razoável: rota 7% pior que o ótimo, mas com resposta quase instantânea.
- O DFS confirmou o comportamento esperado: rápido em memória, mas sem critério de qualidade — produz soluções muito distantes do ótimo.
- O BFS é impraticável para circuitos grandes, mas garante ótimo em grafos pequenos.

6. Arquitectura do Sistema

6.1 Estrutura de ficheiros

```
TurRoute/
├─ app.py                # Servidor Flask + API REST
├─ algorithms/
│   ├─ greedy.py         # Vizinho Mais Próximo
│   ├─ bfs.py            # Busca em Largura
│   ├─ dfs.py            # Busca em Profundidade
│   ├─ astar.py          # A* com heurística MST (Prim)
│   ├─ runner.py         # Dispatcher dos algoritmos
│   └─ test_algorithms.py # Testes dos cenários obrigatórios
├─ data/
│   ├─ cities.py         # 11 cidades com coordenadas reais
│   ├─ distances.py      # Distâncias Haversine
│   ├─ problem_model.py  # TSPState e TSPProblem
│   └─ routes.py         # Rede de voos directos LAM
├─ ui/
│   ├─ index.html        # SPA principal com Chart.js
│   ├─ css/linear.css    # Design system
│   └─ js/
│       ├─ map.js        # Mapa SVG + animação do avião
│       └─ app.js        # Lógica de interface e gráficos
└─ tests/
    ├─ test_integration.py # 14 testes end-to-end
    ├─ test_edge_cases.py  # 17 testes de casos extremos
    └─ test_metrics.py     # 15 testes de métricas
```

6.2 API REST

Método	Endpoint	Descrição
GET	/api/cities	Lista das 11 cidades com coordenadas
GET	/api/border	GeoJSON do contorno de Moçambique
GET	/api/provinces	GeoJSON das províncias
POST	/api/route	Calcula e devolve rotas optimizadas

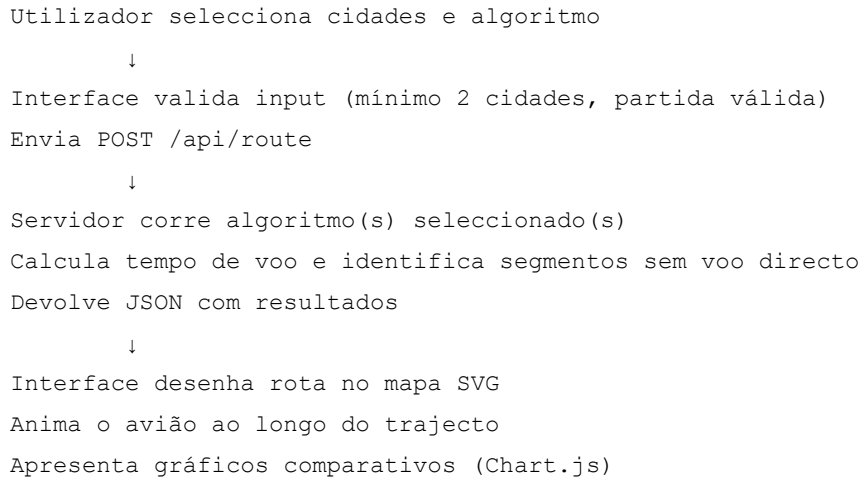
Exemplo de pedido:

```
POST /api/route
{
  "cities": ["Maputo", "Inhambane", "Beira"],
  "start_city": "Maputo",
  "algorithm": "Todos"
}
```

Exemplo de resposta:

```
{
  "results": [...],
  "best": {
    "algorithm": "A*",
    "path": ["Maputo", "Inhambane", "Beira"],
    "cost": 819.8,
    "flight_time_min": 185
  },
  "segments": [
    { "from": "Maputo", "to": "Inhambane", "direct_flight": true }
  ]
}
```

6.3 Fluxo de execução



7. Interface e Visualização

A interface foi desenvolvida como uma SPA (Single Page Application) em HTML, CSS e JavaScript puro, sem dependências de framework front-end.

Funcionalidades principais:

- Mapa SVG de Moçambique com posicionamento geográfico real das cidades
- Animação do avião a percorrer a rota calculada
- Gráficos de comparação entre algoritmos (Chart.js): custo, nós explorados, tempo de execução
- Identificação automática de escalas — quando um segmento não tem voo directo na rede LAM
- Três presets de cenário: Sul, Centro-Norte, Completo
- Adaptação automática da interface conforme o algoritmo seleccionado (um ou todos)

8. Testes e Validação

O sistema inclui **48 testes automatizados** organizados em três ficheiros:

Ficheiro	Nº de testes	Âmbito
test_integration.py	14	Testes end-to-end da API e integração
test_edge_cases.py	17	Casos extremos: 2 cidades, cidades inválidas, BFS com 11+ cidades
test_metrics.py	15	Validação de métricas: distâncias, tempo de voo, formatação

Todos os 48 testes passam. Para executar:

```
python -m pytest tests/ -v
```

9. Divisão de Tarefas

Tarefa	Responsável	Descrição
A	Igor dos Santos	Modelação do problema — estados, acções, custo, representação formal do TSP
B	Stanley Cossa	Interface principal e mapa — layout, mapa SVG, animação do avião
C	Aashir Omar	Visualização de métricas — gráficos comparativos, tabelas, análise automática
D	Saudah Ahmad	Algoritmos de busca — BFS, DFS, Greedy, A* com heurística MST
E	Hélder Júnior	Integração — orquestração entre interface, servidor e resultados
F	Quirson Ngale	Testes, validação e relatório — 48 testes, documentação

10. Conclusão

O TurRoute demonstra que a escolha do algoritmo de busca tem impacto directo e significativo na qualidade da solução encontrada. Para o problema TSP aplicado à rede LAM:

- O ****A*** com heurística **MST**** é a solução correcta quando se pretende o óptimo garantido: explorou apenas 21 estados para encontrar a rota de 2 951 km com 11 cidades.
- O **Greedy** é uma alternativa prática e rápida para cenários onde velocidade importa mais que perfeição.
- O **DFS** confirmou as suas limitações teóricas: sem critério de qualidade, produz soluções muito afastadas do óptimo.
- O **BFS** é adequado apenas para instâncias pequenas, dada a sua complexidade factorial.

A admissibilidade da heurística MST — demonstrada pelo facto de a MST representar o custo mínimo absoluto de conectar as cidades restantes — é a propriedade central que garante a optimalidade do A*. Esta relação entre teoria e resultados empíricos é o contributo principal deste trabalho.

11. Referências

- Russell, S. & Norvig, P. (2020). Artificial Intelligence: A Modern Approach (4ª ed.). Pearson.
- Cormen, T. H., et al. (2009). Introduction to Algorithms (3ª ed.). MIT Press.
- Documentação oficial Flask: <https://flask.palletsprojects.com>
- Documentação Chart.js: <https://www.chartjs.org>
- LAM — Linhas Aéreas de Moçambique, rede de rotas domésticas: <https://www.lam.co.mz>